

Bankai: Ultra-Sparse Adaptation of 1-Bit LLMs via XOR Patches

Nikshep Saravanan
github.com/nikshepsvn/bankai

April 2, 2026

Abstract

True 1-bit large language models have no post-training adaptation method. LoRA, fine-tuning, and quantization-aware training all require continuous weights or gradients that binary models lack. We introduce **Bankai**, the first post-training adaptation method for true 1-bit LLMs, using bitwise XOR operations on binary weights. Bankai patches are sparse XOR bitmasks that modify model weights in-place with a single bitwise operation, incur zero inference overhead, and compress to around one kilobyte.

We validate on Bonsai 8B (PrismML, 2026), a true 1-bit, 8.2 billion parameter language model. Through eight experiments, we demonstrate that: (1) binary MLP weights exhibit massive redundancy; (2) scale-guided bit flips produce $3.88\times$ more behavioral impact than random flips; (3–4) greedy search yields patches that correct specific calculus failures in free generation; (5) patches trained on few probes memorize rather than generalize; and (6) training on diverse probe variations produces patches that **generalize to held-out prompts**—fixing 4 of 17 problems the base model gets wrong (23.5%) with zero breakage on the 13 it already solves.

1 Introduction

The emergence of true 1-bit large language models—where every weight is a single bit—represents a fundamental shift in model efficiency. Bonsai 8B [PrismML, 2026] stores 8.2 billion parameters in 1.15 GB, runs at 44 tokens/second on an iPhone, and achieves benchmark scores competitive with full-precision 8B models.

However, these models present a novel challenge: **they cannot be adapted after deployment**. Every existing method for post-training adaptation requires continuous-valued operations:

- **LoRA** [Hu et al., 2021] adds low-rank weight deltas in float space.
- **Fine-tuning** backpropagates continuous gradients through differentiable weights.
- **Quantization-aware training** adjusts weights during pre-training, not after deployment.

None of these apply to a model whose weights are single bits. Once a 1-bit model is deployed, it is frozen.

We introduce **Bankai**, the first post-training adaptation method for true 1-bit LLMs. The key insight: because weights are bits, the “diff” between two behavioral states is a bitwise XOR mask. Where the mask has a 1, a weight flipped. Where it has a 0, nothing changed. For nearby behavioral variants, this mask is sparse—enabling patches that modify 0.007% of weights (~ 1 KB) with zero inference overhead, microsecond application, and exact reversibility.

We validate Bankai on Bonsai 8B through six experiments, progressing from mechanism validation (Experiments 1–2), through targeted correction (Experiments 3–4), to generalization testing (Experiments 5–6). We find that patches trained on 6 probes memorize specific prompts, but patches trained on 60 diverse probes generalize to held-out problems—fixing 4 of 17 incorrect answers with zero collateral damage.

1.1 Contributions

1. **The first post-training adaptation method for true 1-bit LLMs.** Existing methods (LoRA, fine-tuning, QAT) require continuous weights or gradients and are inapplicable to binary architectures.
2. **Scale-guided targeting.** Per-group FP16 scale factors predict which weight regions are most behaviorally sensitive, enabling $3.88\times$ more efficient search than uniform random sampling.
3. **Layer-level functional specialization in 1-bit architectures.** Early layers (1–4) and the penultimate layer (34) are load-bearing, while middle layers (17–21) contribute minimally.
4. **A characterization of generalization.** Patches trained on few probes memorize; diverse training produces generalization. This is the first empirical characterization of how XOR patches learn.
5. **An open toolkit and patch format** for reproducible creation, application, and evaluation of XOR patches.

2 Background

2.1 True 1-Bit vs. Ternary Models

The distinction between true 1-bit and ternary models is critical. BitNet b1.58 [Wang et al., 2023, Ma et al., 2025] uses ternary weights $\{-1, 0, +1\}$, requiring 2 bits per weight in storage. Despite the “1-bit” branding, XOR on 2-bit ternary encodings produces invalid states: $\text{XOR}(01, 10) = 11$, which has no valid ternary mapping.

Bonsai 8B [PrismML, 2026] is a true 1-bit model where each weight is a single bit: $0 \rightarrow -\text{scale}$, $1 \rightarrow +\text{scale}$. Weights are packed as `uint32` arrays with per-group FP16 scale factors (group size 128). This makes bitwise XOR a semantically valid operation. As of April 2026, Bonsai 8B is the only production-quality true 1-bit LLM.

2.2 Deployment Implications

The properties of XOR patches—kilobyte-scale size, microsecond application, zero inference overhead, instant reversibility—enable a deployment model impossible with existing adaptation methods. A library of domain patches (math, code, medical, legal), each ~ 1 KB, can be hot-swapped at inference time on a phone. The base model stays at 1.15 GB; a thousand patches add 1 MB. LoRA adapters are ~ 100 MB each, add per-token latency, and require weight reloading to swap.

3 Related Work

Low-rank adaptation. LoRA [Hu et al., 2021] and its variants (QLoRA [Dettmers et al., 2023]; DoRA) reduce fine-tuning cost via low-rank weight deltas. These require continuous weights and are not applicable to true 1-bit architectures.

Compact adaptation. RECAST [Xu et al., 2024] reduces task-specific parameters to fewer than 50 via weight reconstruction on continuous-weight models.

Binary neural networks. XOR-Net [Bulat and Tzimiropoulos, 2020] and XNOR-Net [Rastegari et al., 2016] use binary operations for efficient forward passes, not post-training behavioral modification.

1-bit and sub-1-bit LLMs. BitNet [Wang et al., 2023] introduced 1.58-bit LLM training. STBLLM [Dong et al., 2024] pushed compression below 1-bit and observed that some weights in

binarized LLMs can be randomly flipped without significant performance degradation—a finding our Experiment 1 independently confirms.

Bit-flip attacks. Rakin et al. [2019] showed that ~ 20 targeted bit flips can catastrophically degrade a quantized DNN. Subsequent work (T-BFA [Bai et al., 2021]; Versatile Weight Attack) refined targeted attacks. This literature establishes that small numbers of bit flips produce outsized behavioral effects—but focuses exclusively on adversarial degradation. **Bankai inverts this mechanism:** constructive bit flips for targeted capability improvement.

Model editing. ROME [Meng et al., 2022] and MEMIT [Meng et al., 2023] perform targeted factual edits on continuous-weight models. Bankai shares the goal of minimal intervention but operates on binary weights for behavioral (not factual) modification.

4 Methodology

All experiments use Bonsai 8B (`prism-ml/Bonsai-8B-mlx-1bit`), a true 1-bit, 8.2B parameter model based on the Qwen3 architecture (36 layers, 4096 hidden dim, GQA with 32/8 heads). Weights are packed as `uint32` arrays with 1-bit group quantization (`group_size=128`). Experiments were run on Apple M3 (24 GB).

4.1 Logit Gap Probes

We measure behavioral change via **logit gap probes**: pairs of (`correct_token`, `wrong_token`) following a deterministic prompt. The logit gap $G = \text{logit}(\text{correct}) - \text{logit}(\text{wrong})$ is a single-forward-pass measurement: positive means the model prefers the correct answer. We use this as a fast proxy for behavioral change, noting that logit improvement does not always translate to generation improvement (see Section 6).

4.2 Experiment 1: Random Bit Flip Robustness

We flip N random bits across all MLP weight tensors (5.4B bits total) and measure perplexity on a fixed 5-sentence eval set, for $N \in \{100, 1K, 10K, 50K, 100K, 500K\}$ under four strategies: random across all layers, random in layers 16–24, scale-guided medium (25th–75th percentile), and scale-guided high (top 25%).

4.3 Experiment 2: Structured Flips and Layer Specialization

For each of 36 layers, we flip all bits in the entire MLP and measure logit gaps on 8 probes spanning arithmetic, code, geography, and science. We then test row-level flips at varying counts (1–1024 rows) in the most impactful layer and compare high-scale rows vs. random rows at 64 rows.

4.4 Experiment 3: Greedy Patch Search (Arithmetic)

Greedy hill climbing over row-level flips in layers [1, 2, 3, 4, 34] (selected from Experiment 2). Each iteration: sample a row weighted by scale magnitude, flip all 4,096 bits, measure fitness, keep if improved, revert if not. 200 iterations with 6 target probes (arithmetic) and 4 control probes (knowledge).

Fitness function:

$$f = \frac{1}{|T|} \sum_{t \in T} (g_t - g_t^0) - \lambda \cdot \frac{1}{|C|} \sum_{c \in C} \max(0, g_c^0 - g_c) \quad (1)$$

where g_t, g_c are target and control logit gaps, superscript 0 denotes baselines, and $\lambda = 2.0$ penalizes control degradation.

4.5 Experiment 4: Calculus Patch

Same search targeting 6 calculus probes (polynomial differentiation, second derivatives, integrals, primality, trigonometry, exponential derivatives) where the base model fails deterministically (verified across 5 runs). 300 iterations with screening optimization.

4.6 Experiment 5: Variation Testing

We generate 15 novel variations per probe category (90 total) and measure sign flips (wrong→right vs. right→wrong) on the Experiment 4 patch applied to these never-seen probes.

4.7 Experiment 6: Generalization-Optimized Search

Train on 60 probes (10 per category with diverse phrasings), hold out 30 probes (5 per category) for validation. Mean fitness, 300 iterations. Search time scales roughly linearly with probe count: 6 probes → 13 min, 60 probes → 67 min.

4.8 Experiment 7: Patch Stacking

Apply the Experiment 3 math patch (72 flips) and Experiment 4 calculus patch (70 flips) simultaneously via sequential XOR. Verify order independence and reversibility. Test both math and calculus probes.

4.9 Experiment 8: GSM8K Safety Check

Run 50 GSM8K word problems with full generation (400 tokens) and answer extraction. Compare accuracy with and without the Experiment 6 generalized patch to check for collateral damage on general math reasoning.

5 Results

5.1 Experiment 1: Random Flips Are Absorbed

Table 1: Perplexity change under random bit flips in MLP weights.

Flips	% of MLP bits	Max Perplexity Δ
100	0.000002%	< 0.01
10,000	0.0002%	< 0.01
100,000	0.002%	< 0.02
500,000	0.009%	< 0.08

The model is remarkably robust to random perturbation in MLP weights, implying massive redundancy. (Attention weights, embeddings, and the LM head were not tested.)

5.2 Experiment 2: Layer Impact and Scale-Guided Targeting

Scale-guided targeting: At 64 rows in layer 34, high-scale rows produce $3.88\times$ more behavioral impact than random rows (avg $|\Delta_{\text{gap}}|$: 0.459 vs. 0.118). Layer effects appear domain-specific across our probe set—layer 34 shifts physics by +13.8 while shifting code by −15.9.

Table 2: Average absolute logit gap change ($|\Delta\text{gap}|$) when flipping the entire MLP of each layer range. 8 probes across 4 domains.

Layer range	Avg $ \Delta\text{gap} $	Interpretation
0–4 (early)	3.2–7.2	High impact
5–16 (middle)	0.7–3.0	Moderate, decreasing
17–21 (deep mid)	0.7–1.6	Lowest impact
22–33 (late)	1.6–3.4	Moderate, increasing
34 (penultimate)	9.0	Highest impact
35 (final)	3.2	High

5.3 Experiment 3: Arithmetic Patch

200 iterations, 72 accepted flips, 7.5 minutes on M3. The 2+2 probe flips from incorrect (prefers 5) to correct (prefers 4) in both logit gap and free generation. The 7×8 probe improves at the logit level (prefers 56 over 54) but free generation produces 64 ($= 8 \times 8$)—a different wrong answer, illustrating the gap between probe-based and generation-based evaluation.

Patch: 72 flips, 294,912 bits modified (0.005%), 864 bytes. No degradation on 4 measured control probes.

5.4 Experiment 4: Calculus Patch

300 iterations, 70 accepted flips, ~ 13 minutes on M3. Two deterministic failures corrected in free generation:

- $d/dx [x^4 + 3x^2] = 0 \rightarrow 4x^3 + 6x \checkmark$
- The second derivative of x^4 is $12x^3 \rightarrow 12x^2 \checkmark$

Prompt sensitivity finding: The primality probe (Is 97 prime?) is sensitive to a trailing space—the base model’s answer flips from “No” to “Yes” without any patch. This illustrates the fragility of single-token probes and motivates diverse training sets (Experiment 6).

Patch: 70 flips, 286,720 bits (0.005%), 840 bytes. No degradation on 5 measured control probes.

5.5 Experiment 5: Variation Testing (Negative Result)

The Experiment 4 patch was tested on 90 novel variations (15 per category). Sign-flip analysis:

Table 3: Experiment 4 patch applied to 90 never-seen probes.

Metric	Count
Fixed (wrong $\rightarrow$ right)	2
Broke (right $\rightarrow$ wrong)	5
Stayed right	57
Stayed wrong	26
Net sign flips	−3
Base accuracy	62/90 (68.9%)
Patched accuracy	59/90 (65.6%)

Finding: The 6-probe patch memorizes specific prompts rather than shifting capabilities. All 5 broken probes had borderline baseline gaps (< 1.2).

5.6 Experiment 6: Generalization-Optimized Search

Motivated by Experiment 5’s memorization finding, we trained on 60 probes (10 per category with diverse phrasings) and validated on 30 held-out probes.

Table 4: Sign-flip analysis on training and held-out validation sets.

Metric	Training (60)	Validation (30)
Fixed (wrong $\rightarrow$ right)	4	4
Broke (right $\rightarrow$ wrong)	0	0
Stayed right	44	13
Stayed wrong	12	13
Accuracy	44/60 $\rightarrow$ 48/60	13/30 $\rightarrow$ 17/30

Validation fixes span 4 categories: polynomial derivatives, second derivatives, integrals, and primality. Trig and exponential derivative categories saw zero fixes—a calculus-specific layer impact analysis identified layers 5, 6, and 10 as high-impact for calculus but not included in the search set, which may explain the gap.

The base model gets 17 of 30 validation probes wrong; the patch fixes 4 of those 17 (23.5%) while breaking none of the 13 it already solves.

Patch: 93 flips, 380,928 bits (0.007%), 1,116 bytes. No degradation on 5 measured control probes.

5.7 Experiment 7: Patch Stacking

The Experiment 3 math patch (72 flips) and Experiment 4 calculus patch (70 flips) have zero row overlap, yielding 142 combined flips. Stacking is mechanically correct: order-independent (math+calc gives identical results to calc+math) and perfectly reversible (zero drift after removal).

However, behavioral composition shows interference. The math patch damages `poly_deriv` (−1.59) while the calculus patch improves it (+2.77); stacked, they partially cancel (+0.19). Overall: individual patches each fix 2 and break 1 probe; the stacked patch fixes 1 and breaks 0.

Finding: Patches compose algebraically but interfere behaviorally. Stacking is safer (fewer breakages) but less effective (fewer fixes) than individual patches. Joint optimization—searching for flips that improve both domains simultaneously—would likely outperform naive stacking.

5.8 Experiment 8: GSM8K Safety Check

50 GSM8K word problems, generation with answer extraction:

Table 5: GSM8K accuracy with and without patch (50 problems).

	Without patch	With patch
Correct	11/50	14/50
Accuracy	22.0%	28.0%

No degradation detected. The patch slightly improved accuracy (+3 problems), likely within noise for $n = 50$ but directionally positive. Our absolute GSM8K accuracy (22%) is below Bonsai’s reported benchmark (88%) due to evaluation harness differences; the relative comparison is the meaningful signal.

6 Limitations

Evaluation harness limitations. Our GSM8K accuracy (22%) is well below reported benchmarks, indicating evaluation setup differences. Logit gap probes don’t always predict generation outcomes (visible in the 7×8 example). Standard benchmark evaluation with established harnesses is a next step.

Greedy search finds local optima. Population-based evolutionary search with crossover (XOR of XOR patches is a valid patch) could find better solutions.

Row-level granularity is coarse. Each flip modifies 4,096 bits. Per-group (128 bits) or per-bit search could produce more precise patches.

Patch stacking shows interference. Experiment 7 confirms stacking is mechanically sound but behaviorally lossy—individual improvements partially cancel. Joint optimization would likely outperform naive stacking.

Single model. Bonsai 8B is currently the only production-quality true 1-bit LLM. The approach does not extend to ternary models.

Scale factors are not patched. Only binary weights are modified, not FP16 scale/bias values.

Small evaluation set. Our probes cover limited domains.

7 Responsible Use

Bankai modifies model behavior with kilobyte-scale patches invisible at inference time. The same mechanism could insert subtle malicious changes. However, XOR patches are transparent by design: every patch is a readable manifest listing exactly which rows were flipped, and verification is trivial. If patch libraries become a deployment pattern, provenance and verification become critical infrastructure.

8 Future Work

- **Finer granularity:** Group-level (128 bits) or per-bit search for more precise patches.
- **Evolutionary search:** Population-based with crossover via XOR and Hamming-distance diversity pressure.
- **Benchmark evaluation:** MMLU subcategories, GSM8K, HumanEval.
- **Patch stacking:** Empirical composability testing.
- **Cross-model extraction:** XOR between Bonsai variants as naturally occurring patches.
- **Calculus-specific layer selection:** Searching layers 5, 6, and 10 (identified as high-impact for calculus).

9 Conclusion

We have introduced Bankai, the first post-training adaptation method for true 1-bit LLMs. By exploiting the binary nature of weights, behavioral modifications reduce to sparse XOR bitmasks that are orders of magnitude smaller than existing adapters, incur zero inference overhead, and can be applied and reverted in microseconds.

Through eight experiments on Bonsai 8B, we characterized the method’s capabilities and limitations: targeted corrections work reliably, generalization requires diverse training signal, and the overall approach represents a new point in the adaptation design space—one that is uniquely enabled by true binary model architectures.

Code, patches, and all experiments are available at <https://github.com/nikshepsvn/bankai>.

References

- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. *arXiv:2106.09685*, 2021.
- A. S. Rakin, Z. He, and D. Fan. Bit-flip attack: Crushing neural network with progressive bit search. In *ICCV*, 2019.
- P. Dong, et al. STBLLM: Breaking the 1-bit barrier with structured binary LLMs. In *ICLR*, 2025.
- H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv:2310.11453*, 2023.
- S. Ma, H. Wang, S. Huang, et al. BitNet b1.58 2B4T technical report. *arXiv:2504.12285*, 2025.
- PrismML. Bonsai 8B. <https://prismml.com/news/bonsai-8b>, 2026.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT. In *NeurIPS*, 2022.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. In *ICLR*, 2023.
- Y. Bai, et al. Targeted attack against deep neural networks via flipping limited weight bits. In *ICLR*, 2021.
- Y. Xu, et al. RECAST: Reparameterized, compact weight adaptation for sequential tasks. *arXiv:2411.16870*, 2024.
- A. Bulat and G. Tzimiropoulos. XOR-Net: An efficient computation pipeline for binary neural network inference on edge devices. 2020.
- M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *ECCV*, 2016.
- T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. In *NeurIPS*, 2023.